

# 运行时问题及修复

## phase6

### 问题一：应用约束过滤报错

问题在于 `unified_filtered[unified_filtered['feasible']]` 返回的是 pandas DataFrame，不能直接序列化为 JSON。

#### 1. 修复 numpy 类型序列化问题

在 `[constraint_engine.py](file:///Users/zhao/Desktop/Course/研一课程/系统工程/大作业/webapp - v0.2110副本/Utils/constraint_engine.py#L30-L36)` 中修改 `to_dict()` 方法：

```
def to_dict(self):
    return {
        'name': self.name,
        'expression': self.expression,
        'type': self.type,
        'violations': int(self.violations),          # 转换为 Python
        int
        'violation_rate': float(self.violation_rate) # 转换为 Python float
    }
```

#### 2. 修复 DataFrame 序列化问题

在 `phase6.py` 中的两个回调函数：

`apply_constraints` 函数：

```
# 保存到StateManager
state.save('phase6', 'constraints', [c.to_dict() for c in engine.
constraints])
feasible_df = unified_filtered[unified_filtered['feasible']]
feasible_designs = feasible_df.to_dict('records') # 转换为字典列表
state.save('phase6', 'feasible_designs', feasible_designs)
```

`apply_adjusted_constraints` 函数：

```
# 保存可行设计结果集（转换为字典列表以便JSON序列化）
feasible_df = unified_filtered[unified_filtered['feasible']]
feasible_designs = feasible_df.to_dict('records')
state.save('phase6', 'feasible_designs', feasible_designs)
```

## 问题二：约束敏感性分析图不显示

问题原因：

错误 `KeyError: 'cost_total'` 表明数据库中保存了旧的列名 `'cost_total'`，但实际数据使用的是新列名 `'总成本'`。当 `constraint_sensitivity_analysis` 函数尝试使用旧列名访问 `baseline_constraints` 字典时，就会报错。

修复内容：

在 `phase6.py` 中的 `constraint_sensitivity_analysis` 函数中添加了列名映射：

```
# 兼容旧列名（如果数据库中保存了旧值）
constraint_col_mapping = {
    'cost_total': '总成本',
    'perf_coverage': '速度增量',
    'transmit_power': '服务能力'
}

actual_constraint_col = constraint_col_mapping.get(constraint_col,
constraint_col)

if actual_constraint_col not in baseline_constraints:
    # 返回错误提示
    ...

baseline_value = baseline_constraints[actual_constraint_col]
constraint_type = constraint_types[actual_constraint_col]
```

然后将所有使用 `constraint_col` 的地方改为使用 `actual_constraint_col`：

- `unified[actual_constraint_col]` 用于访问数据列
- `constraint_names[actual_constraint_col]` 用于显示图表标题

## 3个功能

### 1. 基于相关性的敏感性分析 (Phase 5)

系统工程的作用：定义设计优先级与边界

在复杂系统设计初期，往往面临成百上千个设计变量。如果对所有变量平均分配资源，会导致工程效率极低。

修改如下：

```
dbc.Col([
    dbc.Card([
        dbc.CardHeader(html.H5("5.4 敏感性分析", className="mb-0")),
```

```

dbc.CardBody([
  dbc.Alert([
    html.Span("🔍", className="me-2"),
    html.Strong("系统工程定量方法: "),
    "识别关键设计变量，量化每个变量对最终MAU值的影响程度",
    html.Br(),
    html.Small("使用龙卷风图直观展示变量的敏感性排序，帮助决策者优先关注高
敏感性变量")
  ], color="info", className="mb-3"),

  dbc.Label("选择目标指标 (Y) "),
  dcc.Dropdown(
    id='select-sensitivity-target',
    options=[],
    placeholder="选择目标指标 (通常为MAU)...",
    className="mb-3"
  ),

  dbc.Label("选择分析变量 (X) - 可多选"),
  dcc.Dropdown(
    id='select-sensitivity-vars',
    options=[],
    multi=True,
    placeholder="选择要分析的设计变量...",
    className="mb-3"
  ),

  dbc.Button([
    html.Span("🚀", className="me-2"),
    "运行敏感性分析"
  ], id='btn-run-sensitivity', color="primary", className='w-100
mb-3'),

  html.Div(id='sensitivity-results', className="mb-3"),

  dcc.Graph(
    id='sensitivity-tornado-plot',
    figure=go.Figure(),
    config={'displayModeBar': True},
    style={'height': '500px'}
  )
])
], className="shadow-sm mb-4")
], md=12)

```

```

@callback(
    [Output('sensitivity-results', 'children'),
     Output('sensitivity-tornado-plot', 'figure')],
    [Input('btn-run-sensitivity', 'n_clicks')],
    [State('select-sensitivity-target', 'value'),
     State('select-sensitivity-vars', 'value')],
    prevent_initial_call=True
)
def run_sensitivity_analysis(n_clicks, target_var, sensitivity_vars):
    """运行敏感性分析 - 识别关键设计变量"""
    if not n_clicks or not target_var or not sensitivity_vars:
        return dbc.Alert("请选择目标指标和分析变量!", color="warning"),
go.Figure()

    try:
        import pandas as pd
        import numpy as np
        import plotly.graph_objects as go

        state = get_state_manager()
        unified = state.load('phase5', 'unified_results')

        if unified is None:
            return dbc.Alert("请先在Phase 5运行批量计算!", color="warning"),
go.Figure()

        if isinstance(unified, list):
            unified = pd.DataFrame(unified)

        if unified.empty:
            return dbc.Alert("数据为空", color="warning"), go.Figure()

        if target_var not in unified.columns:
            return dbc.Alert(f"目标指标 {target_var} 不存在", color="danger"),
go.Figure()

        valid_vars = [v for v in sensitivity_vars if v in unified.columns]
        if not valid_vars:
            return dbc.Alert("选择的分析变量都不存在", color="danger"),
go.Figure()

        base_value = unified[target_var].mean()
        sensitivities = []

        for var in valid_vars:
            var_data = unified[var]

```

```

target_data = unified[target_var]

correlation = var_data.corr(target_data)
std_ratio = (var_data.std() / (var_data.mean() + 1e-6)) if
var_data.mean() != 0 else 0
sensitivity_score = abs(correlation) * std_ratio

sensitivities.append({
    'variable': var,
    'correlation': correlation,
    'std_ratio': std_ratio,
    'sensitivity': sensitivity_score,
    'impact': (var_data.max() - var_data.min()) *
abs(correlation)
})

sensitivities.sort(key=lambda x: x['sensitivity'], reverse=True)

results_card = dbc.Card([
    dbc.CardHeader(html.H5("敏感性分析结果", className="mb-0")),
    dbc.CardBody([
        html.P([
            html.Strong("目标指标: "), target_var, html.Br(),
            html.Strong("基准值: "), f"{base_value:.4f}", html.Br(),
            html.Strong("分析变量数: "), len(valid_vars)
        ], className="mb-3"),
        dbc.Table([
            html.Thead(html.Tr([
                html.Th("变量名"),
                html.Th("相关性"),
                html.Th("标准差比"),
                html.Th("敏感性得分"),
                html.Th("影响范围")
            ])),
            html.Tbody([
                html.Tr([
                    html.Td(s['variable']),
                    html.Td(f"{s['correlation']:.4f}"),
                    html.Td(f"{s['std_ratio']:.4f}"),
                    html.Td(f"{s['sensitivity']:.4f}"),
                    html.Td(f"{s['impact']:.4f}")
                ]) for s in sensitivities
            ])
        ], striped=True, bordered=True, hover=True, size='sm')
    ], color="light")

```

```

fig = go.Figure()

sorted_sensitivities = sorted(sensitivities, key=lambda x:
x['sensitivity'], reverse=True)
y_values = [s['variable'] for s in sorted_sensitivities]
x_values = [s['sensitivity'] for s in sorted_sensitivities]

colors = ['rgba(255, 0, 0, 0.7)' if s['sensitivity'] >
np.median(x_values) else 'rgba(0, 128, 255, 0.7)' for s in
sorted_sensitivities]

fig.add_trace(go.Bar(
    x=x_values,
    y=y_values,
    orientation='h',
    marker=dict(color=colors),
    text=[f"{s['sensitivity']:.4f}" for s in sorted_sensitivities],
    textposition='outside',
    hovertemplate='%{y}<br>敏感性: %{x:.4f}<extra></extra>'
))

fig.update_layout(
    title="龙卷风图 - 设计变量敏感性排序",
    xaxis_title="敏感性得分",
    yaxis_title="设计变量",
    height=400 + len(valid_vars) * 30,
    margin=dict(l=150, r=50, t=50, b=50)
)

return results_card, fig

except Exception as e:
    import traceback
    traceback.print_exc()
    return dbc.Alert(f"敏感性分析失败: {str(e)}", color="danger"),
go.Figure()

```

### 设计逻辑与SE的关联：

该功能通过计算设计变量（X）与目标指标（Y，如MAU）之间的相关系数和标准差比率，量化了灵敏度得分。

龙卷风图（Tornado Plot）的引入是典型的系统工程定量工具，它直观地展示了哪些变量是“主导因素”，哪些是“噪声”。

### 好处：

识别出对最终系统价值（MAU）影响最大的关键设计参数。决策者可以优先投入研发力量优化高

敏感度变量，而对低敏感度变量采取“满足即可”的策略。

通过筛选，可以剔除无关变量，简化后续的仿真和计算模型，降低系统设计的复杂度和计算成本。

## 2. 基于不确定性传播 (Phase 6)

系统工程的作用：实现利益相关者的价值最大化

系统工程不仅关注“名义值（Nominal Value）”，更关注系统在面对现实干扰时的表现。任何设计变量在实际制造和运行中都存在波动。传统设计往往是基于“平均值”或“理想状态”的定性设计，但在实际生产中，环境、磨损、材料属性都是波动的。

修改如下：

```
dbc.Row([
    dbc.Col([
        dbc.Card([
            dbc.CardHeader(html.H5("6.5 不确定性传播分析", className="mb-0")),
            dbc.CardBody([
                dbc.Alert([
                    html.Span("i", className="me-2"),
                    html.Strong("系统工程定量方法："),
                    "使用蒙特卡洛模拟分析输入变量的不确定性如何传播到输出结果",
                    html.Br(),
                    html.Small("量化设计方案的鲁棒性，支持风险决策")
                ], color="info", className="mb-3"),

                dbc.Label("选择目标指标"),
                dcc.Dropdown(
                    id='select-uncertainty-target',
                    options=[],
                    placeholder="选择目标指标（通常为MAU）...",
                    className="mb-3"
                ),

                dbc.Label("选择不确定变量"),
                dcc.Dropdown(
                    id='select-uncertainty-vars',
                    options=[],
                    multi=True,
                    placeholder="选择具有不确定性的设计变量...",
                    className="mb-3"
                ),

                dbc.Label("不确定性分布类型"),
                dbc.RadioItems(
                    id='radio-uncertainty-dist',
```

```

        options=[
            {'label': '正态分布 ( $\pm 5\%$ )', 'value': 'normal'},
            {'label': '均匀分布 ( $\pm 10\%$ )', 'value': 'uniform'}
        ],
        value='normal',
        className="mb-3"
    ),

    dbc.Label("模拟次数"),
    dcc.Slider(
        id='slider-monte-carlo-samples',
        min=100,
        max=5000,
        step=100,
        value=1000,
        marks={i: str(i) for i in range(100, 5001, 1000)},
        tooltip={"placement": "bottom", "always_visible": True},
        className="mb-3"
    ),

    dbc.Button([
        html.Span("🎲", className="me-2"),
        "运行蒙特卡洛模拟"
    ], id='btn-monte-carlo', color="warning", className="w-100 mb-3"),

    html.Div(id='uncertainty-results', className="mb-3"),

    dcc.Graph(id='uncertainty-distribution-plot',
figure=go.Figure(),
                config={'displayModeBar': True})
    ])
], className="shadow-sm mb-4")
])
])

```

```

@callback(
    [Output('uncertainty-results', 'children'),
     Output('uncertainty-distribution-plot', 'figure')],
    Input('btn-monte-carlo', 'n_clicks'),
    [State('select-uncertainty-target', 'value'),
     State('select-uncertainty-vars', 'value'),
     State('radio-uncertainty-dist', 'value'),
     State('slider-monte-carlo-samples', 'value')],
    prevent_initial_call=True
)

```

```

)
def monte_carlo_uncertainty_analysis(n_clicks, target_var, uncertainty_vars,
dist_type, n_samples):
    """蒙特卡洛不确定性传播分析"""
    if not n_clicks or not target_var or not uncertainty_vars:
        return dbc.Alert("请选择目标指标和不确定变量!", color="warning"),
go.Figure()

    try:
        import numpy as np
        import pandas as pd
        import plotly.graph_objects as go
        from scipy import stats

        state = get_state_manager()
        unified = state.load('phase5', 'unified_results')

        if unified is None:
            return dbc.Alert("请先在Phase 5运行批量计算!", color="warning"),
go.Figure()

        if isinstance(unified, list):
            unified = pd.DataFrame(unified)

        if unified.empty:
            return dbc.Alert("数据为空", color="warning"), go.Figure()

        if target_var not in unified.columns:
            return dbc.Alert(f"目标指标 {target_var} 不存在", color="danger"),
go.Figure()

        valid_vars = [v for v in uncertainty_vars if v in unified.columns]
        if not valid_vars:
            return dbc.Alert("选择的不确定变量都不存在", color="danger"),
go.Figure()

        base_target_values = unified[target_var].values
        base_mean = np.mean(base_target_values)
        base_std = np.std(base_target_values)

        all_simulated_values = []

        for var in valid_vars:
            var_values = unified[var].values
            var_mean = np.mean(var_values)
            var_std = np.std(var_values)

```

```

        if dist_type == 'normal':
            uncertainty_pct = 0.05
            simulated_vars = np.random.normal(var_mean, var_std *
uncertainty_pct, n_samples)
        else:
            uncertainty_pct = 0.10
            simulated_vars = np.random.uniform(var_mean * (1 -
uncertainty_pct),
                                                var_mean * (1 +
uncertainty_pct), n_samples)

        correlation = np.corrcoef(var_values, base_target_values)[0, 1]

        simulated_targets = base_mean + correlation * (simulated_vars -
var_mean) * (base_std / (var_std + 1e-6))
        all_simulated_values.extend(simulated_targets)

    all_simulated_values = np.array(all_simulated_values)
    simulated_mean = np.mean(all_simulated_values)
    simulated_std = np.std(all_simulated_values)

    percentiles = [5, 25, 50, 75, 95]
    percentile_values = np.percentile(all_simulated_values, percentiles)

    cv = (simulated_std / abs(simulated_mean)) * 100 if simulated_mean
!= 0 else 0

results_html = dbc.Card([
    dbc.CardBody([
        html.H5("蒙特卡洛模拟结果", className="mb-3"),
        dbc.Row([
            dbc.Col([
                html.Strong("基准均值:"),
                html.P(f"{base_mean:.4f}", className="mb-0"),
            ], md=4),
            dbc.Col([
                html.Strong("模拟均值:"),
                html.P(f"{simulated_mean:.4f}", className="mb-0"),
            ], md=4),
            dbc.Col([
                html.Strong("均值偏差:"),
                html.P(f"{abs(simulated_mean - base_mean):.4f}",
className="mb-0"),
            ], md=4),
        ], className="mb-3"),

```

```

dbc.Row([
    dbc.Col([
        html.Strong("基准标准差:"),
        html.P(f"{base_std:.4f}", className="mb-0"),
    ], md=4),
    dbc.Col([
        html.Strong("模拟标准差:"),
        html.P(f"{simulated_std:.4f}", className="mb-0"),
    ], md=4),
    dbc.Col([
        html.Strong("变异系数:"),
        html.P(f"{cv:.2f}%", className="mb-0"),
    ], md=4),
], className="mb-3"),
html.H6("分位数分布:", className="mb-2"),
dbc.Table([
    html.Thead([
        html.Tr([
            html.Th("5%"),
            html.Th("25%"),
            html.Th("50%"),
            html.Th("75%"),
            html.Th("95%")
        ])
    ]),
    html.Tbody([
        html.Tr([
            html.Td(f"{p:.4f}") for p in percentile_values
        ])
    ])
], striped=True, bordered=True, hover=True, size="sm"),
html.P([
    html.Strong("鲁棒性评估: "),
    "高" if cv < 10 else "中" if cv < 20 else "低"
], className="mt-3 mb-0")
])
], className="shadow-sm")

fig = go.Figure()

fig.add_trace(go.Histogram(
    x=all_simulated_values,
    name='模拟分布',
    nbinsx=50,
    marker_color='rgba(52, 152, 219, 0.7)',
    opacity=0.75

```

```

))

fig.add_vline(
    x=simulated_mean,
    line_dash="dash",
    line_color="red",
    annotation_text=f"均值: {simulated_mean:.4f}",
    annotation_position="top right"
)

for i, (p, val) in enumerate(zip(percentiles, percentile_values)):
    color = 'green' if p == 50 else 'orange'
    fig.add_vline(
        x=val,
        line_dash="dot" if p != 50 else "dash",
        line_color=color,
        line_width=1,
        annotation_text=f"{p}%: {val:.4f}",
        annotation_position="top left" if i % 2 == 0 else "bottom
left",
        annotation_font_size=10
    )

fig.update_layout(
    title=f"不确定性传播分析: {target_var}<br><sub>模拟次数: {n_samples}</sub>
| 不确定变量: {len(valid_vars)}个</sub>",
    xaxis_title=f"{target_var} 值",
    yaxis_title="频数",
    height=500,
    showlegend=True,
    hovermode='x unified'
)

return results_html, fig

except Exception as e:
    import traceback
    print(f"不确定性传播分析失败: {e}")
    print(traceback.format_exc())

fig = go.Figure()
fig.add_annotation(
    text=f"分析失败: {str(e)}",
    xref="paper", yref="paper",
    x=0.5, y=0.5, showarrow=False,
    font=dict(size=16, color="red")
)

```

```

    )
    return dbc.Alert(f"分析失败: {str(e)}", color="danger"), fig

@callback(
    [Output('select-uncertainty-target', 'options'),
     Output('select-uncertainty-vars', 'options')],
    Input('phase6-autoloader', 'n_intervals')
)
def update_uncertainty_options(n_intervals):
    """更新不确定性分析的下拉选项"""
    if not n_intervals:
        return [], []

    try:
        state = get_state_manager()
        unified = state.load('phase5', 'unified_results')

        if unified is None:
            return [], []

        if isinstance(unified, list):
            unified = pd.DataFrame(unified)

        if unified.empty:
            return [], []

        numeric_cols = unified.select_dtypes(include=[np.number]).columns.tolist()

        target_options = [{'label': col, 'value': col} for col in
                           numeric_cols if col in ['MAU', '总成本', '速度增量', '服务能力']]
        var_options = [{'label': col, 'value': col} for col in numeric_cols
                        if col not in ['feasible', 'id']]

        return target_options, var_options

    except Exception as e:
        print(f"更新不确定性选项失败: {e}")
        return [], []

```

### 设计逻辑与SE的关联:

通过蒙特卡洛模拟（Monte Carlo Simulation），将输入的确定性数值替换为概率分布（正态分布或均匀分布）。

通过计算变异系数（CV）和鲁棒性评估指标，该功能量化了“不确定性是如何从输入传导至输出的”。

好处：

传统的平均值分析可能掩盖极端情况。该功能通过分位数（5%-95%）展示，让工程师看到“最坏情况”下的系统表现。

帮助评估设计方案的“抗干扰能力”。一个平均性能略低但波动极小（CV低）的方案，在系统工程中往往优于性能极高但极不稳定的方案，从而支持更加稳健的风险决策。

### 3. 多目标决策支持与偏好建模 (Phase 7)

系统工程的作用：实现利益相关者的价值最大化

系统工程的核心矛盾在于“没有完美的方案，只有最优的折中”。例如，高性能往往意味着高成本。

修改如下：

```
dbc.Col([
    dbc.Card([
        dbc.CardHeader(html.H5("7.3 多目标权衡决策支持", className="mb-0")),
        dbc.CardBody([
            dbc.Alert([
                html.Span("i", className="me-2"),
                html.Strong("系统工程定量方法："),
                "决策区域划分和偏好锥分析，支持多目标权衡决策",
                html.Br(),
                html.Small("通过决策空间可视化理解不同偏好的最优方案选择")
            ], color="info", className="mb-3"),

            dbc.Label("选择决策目标"),
            dbc.Checklist(
                id="checklist-decision-objectives",
                options=[
                    {"label": "MAU效用值", "value": "MAU"},
                    {"label": "总成本 (M$)", "value": "总成本"},
                    {"label": "速度增量 (km/s)", "value": "速度增量"},
                    {"label": "服务能力", "value": "服务能力"}
                ],
                value=["MAU", "总成本"],
                className="mb-3"
            ),

            dbc.Label("决策方法"),
            dbc.RadioItems(
                id="radio-decision-method",
                options=[
                    {"label": "加权求和法", "value": "weighted"},
                    {"label": "ε-约束法", "value": "epsilon"},
                    {"label": "TOPSIS法", "value": "topsis"}
                ]
            )
        ])
    ])
])
```

```

        ],
        value="weighted",
        className="mb-3"
    ),

    dbc.Label("权重分配"),
    html.Div(id="weight-sliders", children=[
        dbc.Row([
            dbc.Col([
                dbc.Label("MAU权重"),
                dcc.Slider(id='slider-weight-mau', min=0, max=1,
step=0.1, value=0.5, marks={round(i, 1): str(round(i, 1)) for i in [0, 0.2,
0.4, 0.6, 0.8, 1.0]}}, className="mb-2")
            ], md=6),
            dbc.Col([
                dbc.Label("成本权重"),
                dcc.Slider(id='slider-weight-cost', min=0, max=1,
step=0.1, value=0.5, marks={round(i, 1): str(round(i, 1)) for i in [0, 0.2,
0.4, 0.6, 0.8, 1.0]}}, className="mb-2")
            ], md=6)
        ])
    ]),

    dbc.Label("偏好锥角度"),
    dcc.Slider(
        id='slider-preference-cone',
        min=10,
        max=90,
        step=5,
        value=45,
        marks={i: str(i) + '°' for i in range(10, 91, 20)},
        className="mb-3"
    ),

    dbc.Label("决策区域划分"),
    dbc.RadioItems(
        id="radio-decision-region",
        options=[
            {"label": "四象限划分", "value": "quadrant"},
            {"label": "六区域划分", "value": "hexagon"},
            {"label": "偏好锥划分", "value": "cone"}
        ],
        value="cone",
        className="mb-3"
    ),

```

```

        dbc.Button([
            html.Span("🔗", className="me-2"),
            "生成决策支持分析"
        ], id="btn-generate-decision-support", color="success",
        className="w-100 mb-3"),

        html.Div(id='decision-support-results')
    ])
], className="shadow-sm mb-4")
], width=12)

```

```

@callback(
    [Output('decision-support-results', 'children'),
     Output('tradespace-plot', 'figure', allow_duplicate=True)],
    Input('btn-generate-decision-support', 'n_clicks'),
    [State('checklist-decision-objectives', 'value'),
     State('radio-decision-method', 'value'),
     State('slider-weight-mau', 'value'),
     State('slider-weight-cost', 'value'),
     State('slider-preference-cone', 'value'),
     State('radio-decision-region', 'value')],
    prevent_initial_call=True
)
def generate_decision_support(n_clicks, objectives, method, weight_mau,
weight_cost, cone_angle, region_type):
    """生成多目标权衡决策支持分析"""
    if not n_clicks or not objectives or len(objectives) < 2:
        return dbc.Alert("请至少选择两个决策目标!", color="warning"),
go.Figure()

    try:
        import numpy as np
        import pandas as pd
        import plotly.graph_objects as go

        state = get_state_manager()
        unified = state.load('phase5', 'unified_results')

        if unified is None:
            return dbc.Alert("请先在Phase 5运行批量计算!", color="warning"),
go.Figure()

        if isinstance(unified, list):
            unified = pd.DataFrame(unified)

```

```

        if unified.empty:
            return dbc.Alert("数据为空", color="warning"), go.Figure()

        valid_objectives = [obj for obj in objectives if obj in
unified.columns]
        if len(valid_objectives) < 2:
            return dbc.Alert("选择的目标指标不存在", color="danger"),
go.Figure()

        obj1, obj2 = valid_objectives[0], valid_objectives[1]

        x_data = unified[obj1].values
        y_data = unified[obj2].values

        x_mean, y_mean = np.mean(x_data), np.mean(y_data)
        x_std, y_std = np.std(x_data), np.std(y_data)

        normalized_x = (x_data - x_mean) / (x_std + 1e-6)
        normalized_y = (y_data - y_mean) / (y_std + 1e-6)

        if method == 'weighted':
            decision_scores = weight_mau * normalized_x + weight_cost *
normalized_y
        elif method == 'epsilon':
            threshold = np.percentile(normalized_x, 75)
            decision_scores = np.where(normalized_x >= threshold,
normalized_y, -np.inf)
        else:
            ideal_point = np.array([np.max(normalized_x),
np.max(normalized_y)])
            negative_ideal = np.array([np.min(normalized_x),
np.min(normalized_y)])

            distances_to_ideal =
np.sqrt(np.sum((np.column_stack([normalized_x, normalized_y]) - ideal_point)
** 2, axis=1))
            distances_to_negative =
np.sqrt(np.sum((np.column_stack([normalized_x, normalized_y]) -
negative_ideal) ** 2, axis=1))
            decision_scores = distances_to_negative / (distances_to_ideal +
distances_to_negative + 1e-6)

        best_idx = np.argmax(decision_scores)
        best_design = unified.iloc[best_idx]

        fig = go.Figure()

```

```

fig.add_trace(go.Scatter(
    x=x_data,
    y=y_data,
    mode='markers',
    name='设计方案',
    marker=dict(
        size=8,
        color=decision_scores,
        colorscale='Viridis',
        showscale=True,
        colorbar=dict(title='决策得分')
    ),
    text=[f"设计ID: {i}<br>{obj1}: {x:.2f}<br>{obj2}: {y:.2f}<br>得分: {s:.3f}"
        for i, x, y, s in zip(range(len(x_data)), x_data, y_data,
decision_scores)],
    hovertemplate='%{text}<extra></extra>'
))

fig.add_trace(go.Scatter(
    x=[x_data[best_idx]],
    y=[y_data[best_idx]],
    mode='markers',
    name='最优方案',
    marker=dict(size=20, color='red', symbol='star',
line=dict(width=2, color='white'))
))

if region_type == 'quadrant':
    fig.add_vline(x=x_mean, line_dash="dash", line_color="gray",
annotation_text="均值X")
    fig.add_hline(y=y_mean, line_dash="dash", line_color="gray",
annotation_text="均值Y")
elif region_type == 'hexagon':
    for angle in range(0, 360, 60):
        rad = np.radians(angle)
        fig.add_shape(type="line",
            x0=x_mean, y0=y_mean,
            x1=x_mean + np.cos(rad) * x_std * 2,
            y1=y_mean + np.sin(rad) * y_std * 2,
            line=dict(color="gray", dash="dot"))
elif region_type == 'cone':
    cone_rad = np.radians(cone_angle)
    cone_length = max(x_std, y_std) * 3

```

```

fig.add_shape(type="line",
              x0=x_mean, y0=y_mean,
              x1=x_mean + np.cos(cone_rad) * cone_length,
              y1=y_mean + np.sin(cone_rad) * cone_length,
              line=dict(color="red", dash="dash", width=2))

fig.add_shape(type="line",
              x0=x_mean, y0=y_mean,
              x1=x_mean + np.cos(-cone_rad) * cone_length,
              y1=y_mean + np.sin(-cone_rad) * cone_length,
              line=dict(color="red", dash="dash", width=2))

fig.update_layout(
    title=f"多目标权衡决策支持: {obj1} vs {obj2}<br><sub>方法: {method}</sub>
| 偏好锥角度: {cone_angle}°</sub>",
    xaxis_title=obj1,
    yaxis_title=obj2,
    height=600,
    hovermode='closest',
    showlegend=True
)

results_html = dbc.Card([
    dbc.CardBody([
        html.H5("决策支持分析结果", className="mb-3"),
        dbc.Row([
            dbc.Col([
                html.Strong("决策方法:"),
                html.P({
                    'weighted': '加权求和法',
                    'epsilon': 'ε-约束法',
                    'topsis': 'TOPSIS法'
                }.get(method, method), className="mb-0"),
            ], md=4),
            dbc.Col([
                html.Strong("决策区域:"),
                html.P({
                    'quadrant': '四象限划分',
                    'hexagon': '六区域划分',
                    'cone': '偏好锥划分'
                }.get(region_type, region_type), className="mb-0"),
            ], md=4),
            dbc.Col([
                html.Strong("偏好锥角度:"),
                html.P(f"{cone_angle}°", className="mb-0"),
            ], md=4),
        ])
    ])

```

```

], className="mb-3"),
html.H6("推荐最优方案:", className="mb-2"),
dbc.Table([
    html.Thead([
        html.Tr([
            html.Th("指标"),
            html.Th("值")
        ])
    ]),
    html.Tbody([
        html.Tr([html.Td("设计ID"), html.Td(str(best_idx))]),
        html.Tr([html.Td(obj1), html.Td(f"
{best_design[obj1]:.4f}")]),
        html.Tr([html.Td(obj2), html.Td(f"
{best_design[obj2]:.4f}")]),
        html.Tr([html.Td("决策得分"), html.Td(f"
{decision_scores[best_idx]:.4f}")])
    ])
], striped=True, bordered=True, hover=True, size="sm"),
html.P([
    html.Strong("决策建议: "),
    "该方案在当前决策方法和偏好设置下表现最优, 建议作为首选方案。"
], className="mt-3 mb-0")
])
], className="shadow-sm")

return results_html, fig

except Exception as e:
    import traceback
    print(f"决策支持分析失败: {e}")
    print(traceback.format_exc())

    fig = go.Figure()
    fig.add_annotation(
        text=f"分析失败: {str(e)}",
        xref="paper", yref="paper",
        x=0.5, y=0.5, showarrow=False,
        font=dict(size=16, color="red")
    )
    return dbc.Alert(f"分析失败: {str(e)}", color="danger"), fig

```

### 设计逻辑与SE的关联:

该功能集成了多种多属性决策方法（加权求和、 $\epsilon$ -约束法、TOPSIS），对应了系统工程中不同的决策逻辑。

权衡空间可视化：通过散点图展示不同方案在两个核心目标（如MAU vs. 成本）上的分布，并标记出当前的“最优方案”。

偏好锥设计：这是一个高级的SE工具，它不仅展示数据，还允许决策者动态调整权重和偏好角度。

**好处：**

它将决策过程从“主观感觉”转向“定量比选”。决策者可以清晰地看到为了多获得性能，系统需要付出多少额外的成本。

在多部门协同的系统工程中，偏好锥和权衡图提供了一个可视化的对话平台，帮助不同利益相关者（如技术部与财务部）在权衡空间内找到彼此都能接受的“帕累托前沿”解。